

MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Balan Gabriel FAF-241

Report

Laboratory Work No. 2

Operating Systems

Topic: Meet the OS You Will Build

Checked by:

Reițman Patricia, *university assistant*

Chișinău – 2026

Contents

1 Initial Setup **3**

2 Part 1 - Building and booting xv6 **3**

3 Part 2 - Building and booting xv6 **3**
3.1 Observations 4

4 Part 3 - Read the source **4**
4.1 Observations 6

5 Part 4 - Write your first xv6 program **6**

6 Wrap-up **6**

7 Conclusion **7**

1 Initial Setup

```
cd ~/xv6-riscv 2>/dev/null || \
  git clone https://github.com/mit-pdos/xv6-riscv ~/xv6-riscv
cd ~/xv6-riscv
ls                      # kernel/  user/  Makefile  README ...
```

Contrary to the laboratory condition, I have installed a completely fresh image, hence nothing has come preinstalled. In order to install the dependencies to run the xv6 OS, I ran the following command:

```
sudo apt update && sudo apt install -y git build-essential gdb-multiarch
↪  qemu-system-misc gcc-riscv64-linux-gnu binutils-riscv64-linux-gnu
```

2 Part 1 - Building and booting xv6

```
make qemu
```

output

```
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M
↪  -smp 3 -nographic -global virtio-mmio.
force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device
↪  virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0
xv6 kernel is booting
hart 2 starting
hart 1 starting
init: starting sh
$
```

We have to run an OS that expects a RISC-V architecture CPU, however we have 2 problems: our environment runs on x86-64 architecture. for this, we use qemu, an emualtor that simualtes a RISC-V environment. After that we run in another issue: normal compilation of the xv6 creates machine code for the x86-64 system, while we run on a RISC-V env. To fix this we use a cross-compiler, that spits out machine code compatible with RISC-V CPU architecture. Hence the crucial parts were: `gcc-riscv64-linux-gnu` and `qemu-system-misc`

3 Part 2 - Building and booting xv6

```
$ ls
$ cat README
```

```
$ echo hello xv6
$ ls | grep c        # pipes work here too
$ wc README
$ usertests -q       # optional: the kernel test suite (takes a while)
```

output

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
(...)
console        3 23 0

xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
↪  #cat README
Version 6 (v6).  xv6 loosely follows the structure and style of v6,
but is implemented for a modern RISC-V multiprocessor using ANSI C.

cat            2 3 36728 # ls | grep -c
echo           2 4 35592

48 336 2441 README # wc

usertests starting
test copyin: OK
test copyout: OK
(...)
test unlinkcwd: OK
ALL TESTS PASSED
```

Running these commands we can see how familiar they feel, as xv6 is the ancestor of linux.

3.1 Observations

3 programs that xv6 comes shipped with is cat - used to poutput file content, echo - used to output text from console input, ls - used to list items. We have used a pipe in the command `ls | grep -c`. In order for it to work we need 2 essential operations: output and input. The xv6 shell is much more primitive and simpler than linux shell.

4 Part 3 - Read the source

```
findredhundred@Ubuntu:~/xv6-riscv$ ls
```

Output

```
LICENSE  Makefile  README  fs.img  kernel  mkfs  test-xv6.py  user
```

The important folders are kernel and user. Those are the heard of an OS.

```
sed -n "1,40p" user/ls.c
sed -n "1,30p" user/cat.c
```

Output:

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"
#include "kernel/fcntl.h"

char *
fmtname(char *path)
{
  static char buf[DIRSIZ + 1];
  char *p;

  // Find first character after last slash.
  for (p = path + strlen(path); p >= path && *p != '/'; p--)
    ;
  p++;
  (...)
```

We notice that this is recognisable C, with functions like read() and write() which represent syscalls. They can be found like this:

```
catuser/user.h        # the list of system calls available toprogramsgr
ep-n "sys_read\|sys_write" kernel/sysfile.c | head
```

```
#define SBRK_ERROR ((char *)-1)
struct stat;
// system calls
int fork(void);
int exit(int) __attribute__((noreturn));
int wait(int *);

69:sys_read(void)
83:sys_write(void)
```

4.1 Observations

While analysing cat.c, I have noticed that the c program makes numerous system calls, including read() - gets user input, exit() - ends the program with a status, write() - writes to the output stream, open() - opens a file and close() - closes a file. The sys_read returns a uint64 and is defined in kernel/sysfile.c, on line 68. The files in the kernel define the instructions that the kernel is able to do, and how it does them, while the files defined in user allow the user to interact with the kernel's implemented behavior using programs.

5 Part 4 - Write your first xv6 program

I have created the file `user/sleep.c`:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));   // system call into the kernel
  exit(0);
}
```

then updated the makefile to compile the new program:

```
UPROGS=\
    $U/_cat\
    $U/_echo\
    $U/_sleep\        # <-- added this line
    ...
```

Now, after rebuilding and booting xv6, we can run the sleep command!

```
$ sleep 10
$ sleep
usage: sleep <ticks>
```

6 Wrap-up

This laboratory work has helped me understand much better the underlying structure of a simple operating system and how its capabilities are linked together. By exploring the file structure of a simple xv6 OS, I have managed to extend it's functionality with a

simple sleep program, hence understanding how the workflow for developing a very simple OS works.

7 Conclusion

This laboratory work provided a practical introduction to xv6 and to the workflow used when developing a small operating system. Building xv6 for RISC-V, running it through QEMU, inspecting its user and kernel source code, and tracing system-call-related functionality showed how user programs depend on services implemented by the kernel. Adding the sleep program also demonstrated the complete path from writing a user-space utility to including it in the xv6 build and executing it inside the operating system.